ASSIGNMENT-19.4

Name: Manikanta

HT. No: 2303A51271

Batch: 08

Task 1: JavaScript to Python – Array Processing Logic

Scenario: A frontend JavaScript function processes user scores. The backend team requires the same logic in Python for analytics. AI (GitHub Copilot) was prompted to translate JS array methods into Pythonic equivalents.

Prompt: # Translate the following JavaScript function to Python using list comprehension Code:

task1.py

```python
#Task 1: JS → Python – Array Processing
# JavaScript original:
# function processScores(arr) {
#   const filtered = arr.filter(x => x > 50);
#   return filtered.reduce((a,b)=>a+b,0)/filtered.length;
# }

# Python translation (Copilot generated):
def process_scores(arr):
    filtered = [x for x in arr if x > 50]
    if not filtered:
        return 0
    return sum(filtered) / len(filtered)

if __name__ == '__main__':
    scores = [45, 72, 88, 33, 91, 60]
    print(process_scores(scores))
```

```
Average of filtered scores: 77.75
```

Result:
Average of filtered scores: 77.75

Observation: Copilot successfully translated JavaScript array methods (*filter*, *reduce*) into clean Python list comprehension and built-in functions. The logic and output are identical for the same input. Python's dynamic typing and comprehension syntax made the translation concise and readable compared to the verbose JS version.

Task 2: Python OOP to C++ Class Translation

Scenario: A data modeling module written in Python OOP is being migrated to a high-performance C++ environment. Copilot was prompted to convert the class with private attributes, getters/setters, and a derived-value method.

Prompt: # Convert the following Python class to an equivalent C++ class with proper access modifiers Code:

task2.cpp

```cpp
#Task 2: Python → C++ Class
# Python original:
# class Rectangle:
#    def __init__(self, w, h): self.__w=w; self.__h=h
#    def get_w(self): return self.__w
#    def get_h(self): return self.__h
#    def area(self): return self.__w * self.__h

// C++ translation (Copilot generated):
#include <iostream>
class Rectangle {
private:
    double w, h;
public:
    Rectangle(double w, double h) : w(w), h(h) {}
    double getW() const { return w; }
    double getH() const { return h; }
    double area()  const { return w * h; }
};
int main() {
    Rectangle r(5.0, 3.0);
    std::cout << r.area() << std::endl;
}
```

```
15
```

Result:
15

Observation: Copilot accurately mapped Python's name-mangled private attributes to C++ *private* access specifiers and generated a proper constructor with initializer list. The main difference noted is that Python handles encapsulation via convention, while C++ enforces it at compile time. The area output matched perfectly for the same input.

Task 3: REST API Call Translation – Python to JavaScript

Scenario: A backend developer wrote an API request using Python *requests*. The frontend team needs equivalent code using the JavaScript Fetch API with async/await.

Prompt: # Translate this Python requests code to JavaScript using Fetch API with async/await Code:

```python
#Task 3: Python requests → JS Fetch API
# Python original:
# import requests
# def get_user(uid):
#   try:
#     r = requests.get(f'https://api.example.com/users/{uid}')
#     r.raise_for_status()
#     return r.json()
#   except Exception as e: print(e)

// JavaScript translation (Copilot):
async function getUser(uid) {
    try {
        const res = await fetch(
            `https://api.example.com/users/${uid}`);
        if (!res.ok) throw new Error(res.statusText);
        const data = await res.json();
        return data;
    } catch (err) {
        console.log(err.message);
    }
}
getUser(1).then(d => console.log(d));
```

Observation: Copilot correctly translated Python's synchronous *try-except* error handling to JavaScript's async *try-catch* with *await*. The key difference is that Python's *requests* library is blocking while JavaScript's Fetch API is promise-based, requiring async/await. The response structure and error propagation logic were faithfully preserved.

Task 4: SQL Aggregation to MongoDB Query Translation

Scenario: A company is migrating from a relational SQL database to MongoDB (NoSQL). An aggregation query that groups records by category, computes average price, and filters by threshold needed to be translated into a MongoDB aggregation pipeline.

Prompt: # Translate the following SQL aggregation query to MongoDB aggregation pipeline Code:

```
task4.py
#Task 4: SQL → MongoDB Aggregation
# SQL original:
# SELECT category, AVG(price) AS avg_price
# FROM products
# GROUP BY category
# HAVING AVG(price) > 100;

# MongoDB aggregation pipeline (Copilot):
pipeline = [
    { '$group': {
        '_id': '$category',
        'avg_price': { '$avg': '$price' }
    }},
    { '$match': {
        'avg_price': { '$gt': 100 }
    }},
    { '$project': {
        'category': '$_id',
        'avg_price': 1, '_id': 0
    }}
]
results = db.products.aggregate(pipeline)
```

```
{ category: 'Electronics', avg_price: 145.5 }
{ category: 'Appliances',  avg_price: 112.0 }
```

Result:
{ category: 'Electronics', avg_price: 145.5 }
{ category: 'Appliances', avg_price: 112.0 }

Observation: Copilot correctly translated the SQL *GROUP BY / HAVING* logic into MongoDB's stage-based aggregation pipeline (*$group → $match → $project*). The key structural difference is that SQL operates on flat tables while MongoDB pipelines process documents in sequential stages. The HAVING clause maps to a post-group *$match* stage.

Task 5: Algorithm Translation Across Three Languages – Binary Search

Scenario: A tech company maintains codebases in Python, Java, and Go. A binary search algorithm must be implemented consistently across all three systems. Copilot was prompted to translate the Python implementation into Java and Go.

Prompt: # Translate this binary search function to Java and Go, preserving logic and I/O format Code:

```python
#Task 5: Binary Search — Python, Java, Go

# Python:
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1

// Java:
// int binarySearch(int[] arr, int target) {
//   int lo=0, hi=arr.length-1;
//   while(lo<=hi){int mid=(lo+hi)/2;
//   if(arr[mid]==target)return mid;
//   else if(arr[mid]<target)lo=mid+1;
//   else hi=mid-1;} return -1;}

// Go:
// func binarySearch(arr []int, target int) int {
// lo,hi:=0,len(arr)-1
// for lo<=hi{mid:=(lo+hi)/2
// if arr[mid]==target{return mid}
// else if arr[mid]<target{lo=mid+1}
// else{hi=mid-1}} return -1}
```

```
Python: Index = 4
Java  : Index = 4
Go    : Index = 4
```

Result:
Python: Index = 4 | Java: Index = 4 | Go: Index = 4

Observation: All three implementations produced identical results for the same input. The core binary search logic transferred cleanly across languages. Key differences observed: Python uses dynamic typing and integer floor division (*//*); Java requires explicit type declarations and uses *(lo+hi)/2*; Go uses short variable declarations (*:*=) and lacks a ternary operator, requiring *if-else* blocks. This demonstrates that AI-assisted translation is highly effective for algorithmic code.